

Apprentissage de bibliothèque défaisable : des méthodes typées à contrats d'exécution qui se désapprennent à la dérive

Nathanael Braun

2026

Préprint v1 — rapport de système / d'expérience

Résumé

Les agents fondés sur de grands modèles de langage réutilisent leur travail passé via une mémoire floue — recherche (RAG), raisonnement à partir de cas, bibliothèques de compétences — qui rappelle par similarité de surface et n'a aucune notion d'une prémisse devenue fausse. Lorsque le monde dérive d'une manière qui ne change pas la requête, ces mémoires continuent de servir une réponse périmée. Nous présentons l'**apprentissage de bibliothèque défaisable** : une bibliothèque apprise de méthodes typées et composables, chacune dotée d'un **contrat d'exécution défaisable**. Une méthode est supposée à la composition, vérifiée à l'exécution, et — lorsque sa postcondition induite échoue — rétractée avec attribution de blâme (un désapprentissage de type JTMS), après quoi la bibliothèque est révisée, non jetée. La structure typée qui rend une méthode canonicalisable rend aussi sa réutilisation amortissable et sa composition vérifiable sur les seuls contrats, à contexte par appel borné. Sur un moteur réel à base de règles, cohérent par maintien de la vérité (JTMS), en isolant chaque mécanisme : (E2) sous une invalidation de prémisse externe en cours de flux, les mémoires par rappel seul servent du périmé, tandis que tout cache doté d'un crochet d'invalidation récupère. Ce qu'un contrat typé déclaratif ajoute à un rappel codé à la main, c'est une éviction sélective et principielle, plus la généralité et la sûreté de composition (confirmé sur modèle réel) ; (E1) le transfert structurel inter-problèmes est correct et gratuit sur des instances apparentées tenues à l'écart, tandis que l'ablation sans transformation est non saine, et le seul nombre d'appels ne permet pas de les distinguer ; (E3) une vérification de composition en boîte fermée coïncide avec le résultat en boîte ouverte sur chaque paire évaluée (aucun faux-admis), chacune des trois barrières de sûreté étant porteuse ; (P4) l'amortissement est un gradient de la fraction canonicalisable, avec sûreté à chaque couverture et non-sûreté au-delà ; (E6) en tête-à-tête face aux systèmes de mémoire d'agents *nommés* (MemGPT, Reflexion, GraphRAG), chacun — dans sa configuration la plus favorable — peut récupérer à la dérive, mais seul le contrat typé récupère au moindre coût sur (appels $\times$ exactitude $\times$ contexte) *simultanément* : le seul bras qui récupère à la dérive tout en restant non-dominé (aucun bras ne l'égale-ou-bat sur les appels ET le contexte par appel tout en récupérant ; les bras moins chers sont sur la frontière de coût mais échouent à la dérive). (E7) à travers une **chaîne** apprise de méthodes à deux maillons, le coin tient et se *renforce* : la péremption d'une mémoire par rappel seul se cumule de maillon en maillon tandis que le contrat typé récupère **les deux** maillons sélectivement — mesuré sur la vue-croyance et confirmé sur un modèle local réel pour le tête-à-tête à deux maillons ; reproduit sur le vrai exécuteur durable (redémarrage et reprise-après-crash) et montré passer à l'échelle en profondeur (péremption $\propto L$, récupération en $O(1)$) sur le simulateur déterministe. (E8) la moitié *réviser* de la boucle — spécialiser la précondition d'une méthode sur blâme — désapprend une affirmation trop générale en un seul blâme puis se stabilise (0 re-blâme, faux-admis $\rightarrow$ 0), là où la seule éviction de cache re-blâme et re-dérive la classe périmée à chaque épisode (coût $\propto K$). Aucun mécanisme n'est

nouveau (JTMS, contrats à blâme, apprentissage de bibliothèque, révision de théorie, logique de séparation) ; la contribution est leur composition en une bibliothèque apprise de méthodes typées qui réalise un désapprentissage principal et sélectif à la dérive — ce que les mémoires par rappel seul ne font pas — bornée par un plafond K1 mesuré.

1 Introduction

Un agent qui résout de nombreux problèmes apparentés devrait devenir moins coûteux et plus fiable avec le temps. La voie dominante consiste à mémoriser et rappeler : stocker des solutions ou compétences passées, retrouver la plus proche, la réutiliser ou l’adapter. La génération augmentée par recherche [19], le raisonnement à partir de cas et les bibliothèques de compétences comme Voyager [33] partagent cette forme et le même angle mort : ils rappellent par similarité de *surface* et ne représentent pas une *prémisse devenue fausse*. Quand le monde change d’une manière qui ne change **pas** la requête — une réglementation se durcit, un fait est audité et trouvé faux, une politique est révoquée — la réponse en cache reste le plus proche voisin, et elle est toujours servie. Les bibliothèques statiques (programmes appris, compétences distillées [14, 7]) ont le problème inverse : saines à l’apprentissage, elles ne savent pas *désapprendre*.

L’ingrédient manquant est un **contrat typé défaisable** sur chaque unité réutilisable. Empruntant aux contrats logiciels avec blâme [17] et à la vérification graduelle [5], une méthode déclare ce qu’elle lit, écrit, exige et garantit, sur un alphabet de faits *typé*. Pour une méthode *apprise* la garantie est une hypothèse induite : **supposée à la composition, vérifiée à l’exécution, rétractée avec blâme** en cas de violation — une opération de maintien de la vérité [11, 10] — après quoi la bibliothèque est révisée en *spécialisant* la précondition fautive, non en la supprimant (révision de théorie [24, 29] ; révision de croyances [3]). La même structure typée rend la réutilisation **amortissable** (une clé canonique stable) et la composition **vérifiable sans ouvrir la boîte** [23, 28]. Le tout est borné par une unique contrainte honnête, K1 : seule la structure typée, canonisable, amortit ; la prose véritable reste dans le modèle.

Nous employons *apprentissage de bibliothèque* au sens de DreamCoder/Stitch [14, 7] (induire des méthodes typées réutilisables par abstraction [26]), non un ajustement statistique ; on pourrait parler d’*induction de méthodes*. La nouveauté n’est **pas** l’induction mais le **contrat défaisable** qui permet de *désapprendre* une méthode induite. Le changement est de flot de contrôle : là où une mémoire par similarité fait *requête* → *retrouver* → *réutiliser*, la nôtre fait *requête* → *retrouver-le-contrat* → *vérifier* → *exécuter* → *contrôler* → *rétracter* → *spécialiser*. Le suffixe contrôler-et-rétracter est ce que les expériences isolent.

Contributions. (1) Le cadrage des méthodes d’agent réutilisables comme **non-terminaux typés à deux faces dotés d’un contrat d’exécution défaisable**, et la boucle supposer/vérifier/rétracter-blâmer/réviser. (2) Une évaluation reproductible, isolant les mécanismes, sur un moteur réel — la mémoire par rappel seul ne peut pas désapprendre tandis qu’un contrat typé déclaratif récupère la dérive *sélectivement, généralement, sûrement-en-composition* (E2, simu + réel, avec une référence Invalidant équitable) ; transfert structurel sain que le seul nombre d’appels ne certifie pas (E1) ; aucun faux-admis sur les compositions évaluées, chacune des trois barrières ablée (E3) ; amortissement en gradient de la fraction canonisable (P4) — en explicitant ce que chaque expérience établit ou non.

2 Approche

2.1 L’objet : une méthode à deux faces

Une **méthode** est, pour son appelant, une boîte noire à contrat typé ; à l’intérieur, des *productions* la réalisent (séquence, branchement, map, fold). Formellement, un non-terminal de remplacement d’hyperarêtes [18, 12] à sélection conditionnée par précondition [15]. Nous tenons deux régimes séparés par *intention de conception* (nous ne prouvons pas la décidabilité ici). La **grammaire des méthodes** est *censée* rester décidable, via un rang de montage bien fondé et des invariants de typage ; l’existence d’un plan HTN récursif étant indécidable en général [16], nous nous restreignons au fragment bien fondé. L’**exécution** sur des données de taille d’exécution est l’inverse : une couche explicitement bornée par budget, Turing-complète. Le lien à la logique monadique du second ordre [9] est une motivation, non un théorème établi ici.

2.2 Le contrat : un triplet de séparation défaisable

Une méthode déclare une empreinte de lecture, une empreinte d’écriture, une précondition, une postcondition et une étiquette d’effet. La composition sous état partagé est le problème du cadre [20] ; nous le levons par une discipline d’empreintes de logique de séparation [23, 28] sur l’alphabet typé fini. Pour une méthode *apprise* la postcondition est une hypothèse induite : **supposée à la composition, vérifiée à la stabilisation, rétractée avec blâme** [17, 5]. Le moniteur d’exécution est le JTMS [11], donnant une sûreté *éventuelle* (non statique) — précisément le désapprentissage qui manque aux références. Ce désapprentissage est symbolique (une rétractation de croyance par JTMS) et distinct du *machine unlearning* paramétrique [6], qui efface l’influence de données d’entraînement des poids d’un modèle.

2.3 Pipeline et le plancher K1

Une formulation est typée en un but ; une méthode est sélectionnée et composée sur contrats ; les cas la traversent ; les traces distillent (anti-unification [26] ; filtrées par MDL [14, 7]) en de nouvelles méthodes typées ; la dérive rétracte. Le repli universel est le *plancher de micro-tâches* : tout ce qui ne se réduit pas à une méthode typée en cache se réduit à une micro-tâche qu’un petit modèle traite aisément. Un contrat *manquant* coûte un appel modèle bon marché ; un contrat *faux* est rattrapé par la vérification d’exécution — jamais une erreur silencieuse.

3 Mise en œuvre

Tous les mécanismes sont réalisés sur un moteur existant de graphe de faits typés à base de règles, cohérent par JTMS, à concepts déclaratifs et stabilisation par chaînage avant, sans modification du cœur hormis une option de requête additive. La clé de mémorisation typée est le condensé de canonicalisation du moteur ; le vérificateur de contrat défaisable (entailment à la composition sur domaines abstraits, assertion de postcondition à l’exécution, trois barrières de sûreté) et la transformation de transfert structurel (relativiser-au-stockage / lier-au-rejeu) sont des bibliothèques hôtes. Un exécuter de cas durable à l’échelle est un artefact connu (un réseau de workflow [32] sur un magasin durable, dans la lignée d’AWS Step Functions Distributed Map [4], du cache de contenu de Prefect, et de DBOS [31]) ; notre vue-croyance se situe au-dessus. Le cycle de vie défaisable — **supposer** → **vérifier** → **contrôler** → **rétracter** → **spécialiser** — correspond un-à-un aux fonctions du moteur :

```

select(but): # SUPPOSER (a la composition)
  M <- bibliotheque.match(but.faits_typed) # cle typee ; un echec -> plancher de micro-taches
  supposer M.contrat # checkCompose: post(prec) |= pre(M); escalade, jamais faux-admis

apply(M, cas): # VERIFIER + CONTROLER (a l'execution)
  cle <- digest(cas.premisse_typee) # cle canonique K1
  si memo.has(cle): retourner memo[cle] # amortir un cas type recurrent
  sortie <- run(M, cas) # sinon derivier (appel modele / sous-graphe)
  si non assertPost(M.contrat, sortie): # la post tient ? + G1 cadre + G2 oracle-d'effet
    quarantaine(cas); blamer(M.contrat) # ne jamais valider une mauvaise sortie
  retourner
  memo[cle] <- sortie; retourner sortie

on ingest(fait): # RETRACTER + SPECIALISER (derive)
  pour e dans memo dependant de fait: # JTMS: re-verifier chaque post affectee
    si non satisfies(e.contrat.post, e.faits + {fait}):
      retracter(e); blamer(e.contrat) # desapprendre: evincer + attribuer le blame
  bibliotheque.reviser(blame): pre <- specialiser(pre) # reviseOnBlame (CEGIS): restreindre, sans
    supprimer

```

4 Expériences

4.1 Dispositif

Les expériences utilisent les mécanismes réels du moteur à deux fidélités explicites. **E1 et E3instancient le moteur complet** (graphe + stabilisation + JTMS) ; **E2, P4 et E5 isolent les fonctions** réelles (`digest`, `satisfies`, `canonValue`) depuis un banc, sans la boucle complète — nous ne prétendons pas qu'E2 exerce la rétractation JTMS native (il la ré-implémente sur le même prédicat réel). Le modèle est un **simulateur déterministe** (oracle parfait de la règle *courante* étant donné seulement ce que révèle l'invite — toute péremption/coût vient du mécanisme, non d'une erreur modèle) ou un **modèle local réel** (Qwen3.6-27B (Q2_K_XL, MTP)). Chaque exécution est conditionnée par un **auto-test du banc** (sous le simulateur le bras naïf doit être parfait, sinon avortement). Sept bras partagent une interface : Naïf, Long-contexte, RAG, CBR (clé typée, sans re-vérification), Compétence (prose), **Invalidant** (cache à clé typée + rappel grossier codé à la main : crochet d'invalidation, sans contrat typé), et **Struct** (la bibliothèque typée au contrat défaisable). L'Invalidant sépare « possède une invalidation » de « possède un contrat typé ».

4.2 E2 — défaisance à la dérive

Un domaine d'approbation typé ($N=80$, deux classes auditées ; l'exécution réelle utilise $N=48$, une classe) avec une invalidation de prémisses *externe* en cours de flux : un audit de conformité marque une classe non conforme, faisant basculer ses cas approuvés vers le refus. L'audit n'est **pas un champ d'enregistrement**, donc un cache par rappel seul retrouve le même enregistrement et sert sa réponse pré-audit (Table 1).

La lecture est triple. Les **mémoires par rappel seul servent du périmé** — le rappel seul ne récupère pas, l'audit n'entrant jamais dans leur chemin de réutilisation. La **récupération exige un mécanisme d'invalidation**, et l'Invalidant comme Struct en ont un (exactitude-dérive 1.00). Ce que le **contrat défaisable typé ajoute au rappel codé à la main** : (i) la *sélectivité* — Struct re-vérifie la post par entrée (`satisfies`) et n'évince que les 2 classes *violées* (approve), là où le rappel jette des classes entières (4 entrées) et paie les re-dérivations supplémentaires (26 vs 28

bras	appels	exact.	exact. dérive	ctx max
Struct (contrat typé)	26	1.00	1.00	290
Invalidant (crochet, sans contrat)	28	1.00	1.00	290
Naïf	80	1.00	1.00	290
Long-contexte	80	1.00	1.00	2062
RAG	48	0.95	0.00	290
CBR (clé typée, sans re-vérif.)	24	0.95	0.00	290
Compétence (prose)	80	0.95	0.00	297

Table 1: E2 (simu). Les bras par rappel seul (RAG/CBR/Compétence) servent du périmé ; Invalidant et Struct récupèrent.

appels) ; (ii) la *généralité* — le même **assertPost**, agnostique-à-la-prémisse par construction (les expériences n'exercent qu'un seul type de prémisse, un basculement de drapeau de conformité) ; (iii) la *sûreté de composition* (§4.4). L'exécution réelle (Qwen3.6-27B (Q2_K_XL, MTP), $N=48$) reproduit ceci : RAG/CBR/Compétence 0.00 ; Invalidant 14 appels / 1.00 ; Struct 13 appels / 2.6 s / 1.00 / ctx 278 vs Long-contexte 1304. L'affirmation défendable n'est pas « seul Struct récupère » mais « la mémoire par rappel seul ne sait pas désapprendre, et un contrat typé déclaratif fournit la récupération de façon sélective, générale et sûre en composition ».

4.3 E1 — amortissement et transfert structurel

Un domaine de décomposition structurelle (une méthode qui *crée* un sous-graphe avec des identifiants), sur le moteur complet. Partition : entraînement, apparentés tenus à l'écart, nouveau tenu à l'écart. C'est un contrôle d'**existence-et-sûreté sur un petit ensemble** (2 apparentés, 1 nouveau), **pas un taux** : avec la transformation relativiser/liar, *toutes* les instances apparentées transfèrent à 0 appel et **sainement**, le nouveau paie (pas de faux rejeu), totaux 3 appels contre 5. L'ablation sans transformation « touche » mais rejoue le *mauvais espace d'identifiants* — **non sain**. **Seule la vérification de sûreté** distingue une réutilisation saine d'un rejeu erroné ; le seul nombre d'appels les classe à égalité. (Un *taux* de transfert sur de nombreuses méthodes est un travail futur.)

4.4 E3 — sûreté de composition

En composant des paires sur leurs seuls contrats typés (boîte fermée) et en comparant au résultat moteur (moteur complet), la décision **coïncide avec la réalité sur chaque paire évaluée, sans faux-admis** — le vérificateur n'accepte jamais à tort ; les paires sous-déterminées ou hors-fragment *escaladent*. C'est montré sur un petit ensemble construit (3 paires sain/non-sain/escalade ; une 4^e ajoute l'oracle) : une **démonstration d'existence**, pas un taux de population. Chaque barrière est porteuse sur un exemple : retirer la complétude de cadre manque une écriture non déclarée ; retirer l'étiquette d'effet admet un effet externe non vérifié ; retirer la détection de cycle admet un cycle couplé rétractable. La décision ne lit que l'empreinte partagée ; le vérificateur (entailment par domaines abstraits, sain-mais-incomplet) est l'artefact le plus développé, plutôt sous-évalué ici. (Un corpus plus grand, non trié à la main, est un travail futur.)

4.5 P4 — le plafond de couverture K1

Sur une charge mixte (fraction p entièrement typée ; le reste portant un composant en prose qui prime sur la règle typée), l'appartenance à K1 décidée par la **vraie** barrière de canonicalisation, l'amortissement est un **gradient en couverture** (approbation : $0 \rightarrow 19 \rightarrow 44 \rightarrow 69 \rightarrow 94\%$; tri : $0 \rightarrow 22 \rightarrow 47 \rightarrow 72 \rightarrow 97\%$). L'exactitude de Struct est 1.00 à chaque couverture — la fraction non typée est un *coût* de micro-tâche, jamais une *falaise* de sûreté. Une variante gloutonne mémorisant les enregistrements en prose sur leur clé typée chute à une exactitude égale à la fraction propre : amortir au-delà de K1 est non sain. Nous sommes explicites : c'est une **illustration construite**, pas une mesure réelle — nous *fixons* p et *définissons* les enregistrements en prose pour primer, donc « non sain au-delà de K1 » découle par construction. Elle établit la *forme* et la sûreté à chaque niveau ; la fraction canonicalisable d'un corpus réel est dépendante du domaine et non mesurée ici.

4.6 E5 — coût de bookkeeping à l'échelle du flux

Un contrôle de coût de bookkeeping, pas une affirmation sur le passage à l'échelle de la partie difficile : sur un espace typé de 200 classes avec un audit, quand la longueur du flux N croît de $1,320 \rightarrow 20,320$ (l'ensemble des classes est fixe ; aucune méthode nouvelle, aucun modèle), Table 2.

N	appels Struct	appels/ N	appels Naïf	bibliothèque	évincés
1,320	202	0,153	1,320	200	2
5,320	202	0,038	5,320	200	2
20,320	202	0,010	20,320	200	2

Table 2: E5. Les appels de Struct restent constants ; bibliothèque bornée par l'ensemble (fixe) des classes ; éviction sélective.

Les appels de Struct restent constants ($\rightarrow$ appels/ $N \rightarrow 0$ tandis que Naïf reste à un) ; la bibliothèque est bornée par le nombre de classes (trivialement) ; une dérive ne rétracte que les classes invalidées ($O(\text{invalidé})$: 2 sur 200). Coûts par opération : canonicalisation de quelques μs /appel (dépend de l'environnement), une passe d'éviction $\approx 0,5\text{ms}$. Le contenu honnête est étroit : le bookkeeping typé n'est pas le goulet. Il ne teste **pas** l'échelle dans la dimension qui compte (bibliothèque croissante de méthodes *distinctes*, corpus réel, modèle réel sur tous les bras) — travail futur.

4.7 E6 — tête-à-tête face aux systèmes de mémoire d'agents nommés

Les références du Dispositif sont génériques (RAG/CBR/Compétence) ; les systèmes attendus en comparaison sont les systèmes *nommés*. Nous ajoutons une ré-implémentation minimale fidèle de MemGPT/Letta [25] (mémoire à étages auto-éditée), Reflexion [30] (réflexion verbale pilotée par l'échec, sans mémo) et GraphRAG [13] (index de résumés de communautés hors-ligne) derrière la même interface, chacun dans sa configuration *la plus favorable* avec une ablation appariée (le contrôle négatif). Stub déterministe, $N = 78$, deux classes auditées (Table 3).

La lecture honnête n'est *pas* « seul Struct récupère » : dans sa configuration la plus favorable, chaque système nommé peut récupérer à la dérive. Le propos est que Struct récupère au moindre coût sur les trois axes à la fois — c'est le seul bras qui récupère à la dérive tout en restant non-dominé (aucun bras ne l'égale-ou-bat sur les appels ET le contexte par appel tout en récupérant ; les bras moins chers sont sur la frontière de coût mais échouent à la dérive). Chaque système nommé paie une taxe propre : MemGPT ne récupère qu'en paginant l'audit fait-surface en mémoire core (un

bras	appels	exact.	exact.-dérive	ctx max
Naïf	78	1,00	1,00	290
MemGPT (audit paginé en core)	23	1,00	1,00	320
—pagination (ablation)	18	0,92	0,00	258
Reflexion (signal d’échec différé)	82	1,00	1,00	332
—signal (ablation)	78	0,92	0,00	258
GraphRAG (index hors-ligne)	87	0,92	0,00	336
+ré-index (ablation)	89	1,00	1,00	336
Struct	20	1,00	1,00	290

Table 3: E6. Dans sa configuration la plus favorable chaque système nommé peut récupérer ; seul Struct récupère à la dérive tout en restant non-dominé sur (appels, exact.-dérive, contexte).

tour modèle), puis *grossièrement* (un drapeau de classe entière re-décidant même les membres non concernés) avec un bloc core dans chaque invite ; Reflexion n’a *aucun* mémo adressé par contenu, donc émet un appel modèle sur *chaque* enregistrement (appels $\approx N$) et ne récupère que réactivement après un échec observé ; l’index hors-ligne de GraphRAG est aveugle à l’audit silencieux (périmé), ne récupérant que par un *ré-résumé par lot* (grossier, hors-bande), jamais un défaiseur par entrée. Les systèmes nommés n’ont pas été conçus pour amortir des décisions ponctuelles typées récurrentes ; cette charge les éprouve hors de leur domaine de conception, de sorte que la « taxe » est le coût d’un outil inadapté, non une infériorité à leur propre tâche. Les ablations confirment que chaque mécanisme est porteur (éteint $\rightarrow$ exact.-dérive 0,00). En isolation, la taxe de récupération de Struct est la re-dérivation des seules entrées *violées* (sélectif) et la ré-assertion du contrat est intra-moteur (**zéro** appel modèle). Un essai en direct (Qwen3.6-27B (Q2_K_XL, MTP), $N = 32$) reproduit l’ordre : Struct 9 appels / 1,9s, MemGPT 11 / 2,3s, Reflexion 34 / 7,1s, GraphRAG 36 / 7,7s (périmé). Ce sont des ré-implémentations minimales fidèles, pas les systèmes complets.

4.8 E7 — composition à la dérive

E6 mesure une *seule* méthode. Le différenciateur d’une *bibliothèque* est la composition : lorsque la prémisse d’une méthode amont tombe, la récupération se propage-t-elle dans la chaîne ? Nous étendons la charge à une chaîne à deux maillons — **decide** $\rightarrow$ **disburse**, où **disbursement** = **disbursed** iff **decision** == **approve** (le maillon 2 lit le fait-résultat du maillon 1) — et ré-exécutons le tête-à-tête en notant les **deux** maillons. Le même audit exogène se propage désormais en cascade : un cas audité à score élevé doit faire basculer **decision** approve $\rightarrow$ reject et **disbursement** disbursed $\rightarrow$ held. Simulateur déterministe ($N=78$, deux classes auditées) et une exécution réelle (Qwen3.6-27B (Q2_K_XL, MTP), $N=48$), Table 4.

Trois choses sont mesurées. (i) **La préemption se cumule.** Tout bras par rappel seul ou aveugle (CBR, et l’ablation de chaque système nommé) est faux au maillon 1 *et* au maillon 2 : une réponse amont périmée empoisonne le maillon aval qui la lit. L’exactitude à la dérive est mesurée sur l’oracle déterministe ; l’exécution réelle confirme l’ordre des appels et du temps (le modèle réel est imparfait à petit N , nous ne lisons donc pas la dérive par enregistrement sur lui). (ii) **La taxe de récupération se multiplie le long de la chaîne.** Reflexion, sans mémo, paie un appel par enregistrement *par maillon* (appels $\approx 2N$) ; GraphRAG ré-indexe les communautés concernées à *chaque* maillon ; MemGPT paie la pagination + un re-décodage grossier + un contexte plus grand aux *deux* maillons. (iii) **La cascade de Struct est sélective.** Une prémisse tombée dé-cast la croyance amont et le changement se propage à l’aval — récupérant les deux maillons

bras	appels (simu / réel)	exact.-dérive maillon 1	exact.-dérive maillon 2
Naïf	156 / 96	1,00	1,00
CBR (= Struct – contrat)	36 / 24	0,00	0,00
MemGPT (le plus favorable)	45 / 41	1,00	1,00
Reflexion (le plus favorable)	160 / 108	1,00	1,00
GraphRAG + ré-index	178 / 116	1,00	1,00
Struct	38 / 26	1,00	1,00
Struct (moteur complet)	38 / 27	1,00	1,00

Table 4: E7 (simu / réel). Chaîne à deux maillons **decide** → **disburse** : la péremption par rappel seul se cumule aux deux maillons ; Struct récupère les deux sélectivement et est l’unique point Pareto-optimal parmi les bras corrects-à-la-dérive sur (appels, exact.-dérive maillon 1, exact.-dérive maillon 2, contexte).

tout en re-dérivant *seulement* l’entrée amont violée (la re-dérivation aval est élidée car sa clé de cache est l’ensemble de lecture réel de disburse {**kind**, **region**, **decision**}). Struct est l’unique point Pareto-optimal parmi les bras corrects-à-la-dérive sur (appels × exact.-dérive maillon 1 × exact.-dérive maillon 2 × contexte par appel), en simu comme en réel ; la réalisation sur moteur complet (quatre concepts *ensure*-gated au-dessus du cache de dérivation) la reproduit (38=38 en simu ; 26 ≈ 27 en réel, dans le non-déterminisme du modèle). C’est la capacité qui manque structurellement aux mémoires de surface nommées : une croyance qui dépend d’une prémisse qui vient de tomber, désapprise à *travers la composition*.

Le coin s’élargit avec la profondeur de chaîne. En généralisant la chaîne à L maillons (chacun positif ssi le précédent l’est), la péremption et le coût croissent tous deux tandis que la récupération de Struct, non : la *profondeur de cumul* d’un cache par rappel seul (maillons faux sur une classe dérivée) vaut exactement L ; Naïf et Reflexion paient $O(L \cdot N)$ appels ; mais la **taxe de dérive** de la récupération de Struct est en $O(1)$ en L — la cascade ne re-dérive que l’entrée amont violée, chaque re-dérivation aval réutilisant l’entrée d’ensemble-de-lecture d’un frère. Ainsi, plus la chaîne de méthodes apprises est profonde, plus l’avantage de Struct est grand sur l’écart d’exactitude (cumul $\propto L$) et sur l’efficacité de récupération ($O(1)$ contre une reconstruction grossière en $O(L)$). (La récupération en $O(1)$ tient lorsque chaque re-dérivation aval retombe sur une clé de cache qu’un frère a déjà remplie — vrai ici : la branche négative est partagée et les frères à faible score la pré-remplissent ; sous un fort éventail aval, la taxe de récupération croît vers $O(L)$. Le cumul $\propto L$ découle par construction du fait de définir le maillon i comme une fonction du maillon $i - 1$, et est mesuré sur le simulateur déterministe — le balayage en L en réel est confondu par le bruit, §6.)

Sur le vrai exécuteur durable. Ce qui précède est la vue-croyance (le graphe à base de règles + la rétractation JTMS). La même chaîne compilée en un réseau de workflow et exécutée comme un flot de jetons sur un magasin de points de reprise adressé par contenu et reprenable après crash reproduit le résultat sur la couche d’*exécution* : la chaîne amortit et la dérive se propage en cascade à travers les deux maillons (Struct-sur-exécuteur 24 appels, dérive 1,00/1,00 ; la clé sans-prémisse comme le cache composé plat y cumulent la péremption), la bibliothèque composée chaude **se rejoue à travers un redémarrage de processus à zéro appel modèle**, et une chaîne coupée en plein vol **reprend sans travail perdu ni dupliqué**. La défaisance en vue-croyance et l’exécution durable sont complémentaires : le contrat rétracte la croyance ; l’exécuteur rend le recalcul durable et sélectif.

4.9 E8 — révision de bibliothèque sous dérive récurrente

E2/E6/E7 mesurent le chemin *rétracter* $\rightarrow$ *re-dériver* : une post violée évince la croyance en cache, qui est ensuite re-déivée. L'autre moitié de la boucle — *réviser* : spécialiser la précondition de la méthode sur blâme, de sorte que la bibliothèque soit améliorée, non simplement ré-évaluée — est évaluée ici. Nous exécutons $K=5$ épisodes de dérive récurrente (les mêmes classes reviennent à chaque épisode après qu'une précondition trop générale a été exposée) et comparons l'éviction-seule à la révision via le `reviseOnBlame` du moteur (Table 5).

sur $K=5$ épisodes	Éviction-seule	Révision (<code>reviseOnBlame</code>)
blâmes (par épisode $\rightarrow$ cumulatif)	1,1,1,1,1 $\rightarrow$ 5	1,0,0,0,0 $\rightarrow$ 1
re-dérivations (appels modèle)	4,1,1,1,1 $\rightarrow$ 8	4,1,0,0,0 $\rightarrow$ 5
taux de faux-admis	0,25 à chaque épisode	0,25 $\rightarrow$ 0,00 après e1

Table 5: E8. L'éviction-seule re-blâme et re-dérive la classe périmée à chaque épisode (coût $\propto K$) ; la révision spécialise la précondition une fois puis se stabilise (0 re-blâme, faux-admis $\rightarrow$ 0).

L'éviction-seule conserve la règle trop générale, donc elle ré-admet, re-blâme et re-dérive la même classe périmée à *chaque* épisode (coût $\propto K$). La révision spécialise la précondition une seule fois — en la restreignant avec l'atome discriminant du contre-exemple — puis se stabilise : plus aucun blâme, faux-admis $\rightarrow$ 0. La révision est *chirurgicale* : la précondition spécialisée exclut la classe fautive tout en admettant encore son frère (elle restreint, elle ne supprime pas la méthode). L'effet tient pour deux types de prémisses — un basculement de conformité catégoriel (`$compliant != false`) et une porte numérique (`$score != 680`). **Limite honnête** : `reviseOnBlame` spécialise par exclusion ponctuelle de contre-exemple, non par resserrement de borne ; une dérive numérique couvrant D valeurs distinctes converge en D blâmes ponctuels (bornés, par valeur) plutôt qu'en un seul déplacement de borne — toujours catégoriquement mieux que la récurrence par épisode de l'éviction-seule. C'est l'étape qui distingue le contrat de l'invalidation de cache (§5) : le blâme alimente la *révision de bibliothèque*, non le seul recalcul de valeurs.

5 Travaux apparentés

Recherche et mémoire de cas. RAG [19] et CBR [1] rappellent par similarité ; ils ne représentent pas une *prémisse devenant invalide* (E2). Les bibliothèques de compétences (Voyager [33]) stockent de la *prose* sans prémisse défaisable.

Mémoire des agents LLM. MemGPT/Letta [25], Reflexion [30], GraphRAG [13] gèrent *quoi* garder/rappeler finement, mais par pertinence/récence/similarité ; aucun ne représente une prémisse typée dont la falsification rétracte une réutilisation. Ils sont complémentaires : un contrat défaisable pourrait s'y placer comme couche de rétractation. Nous menons ce tête-à-tête en E6 (§4.7) : chacun, dans sa configuration la plus favorable, peut récupérer, mais seul le contrat défaisable le fait au moindre coût sur (appels $\times$ exactitude $\times$ contexte) simultanément.

Apprentissage de bibliothèque / EBL. DreamCoder [14] et Stitch [7] font croître une bibliothèque par abstraction [26] ; l'EBG [21] spécialise depuis une seule preuve ; aucun n'attache de contrat d'exécution défaisable.

Contrats, blâme, vérification graduelle. [17, 5] sont la lignée de notre discipline, appliquée aux méthodes *appprises*.

Composition. Non-terminal HRG à deux faces [18, 12, 9] à sélection HTN [15] ; plan HTN récursif indécidable [16] ; composition saine via empreintes de séparation [23, 28], problème du

cadre [20] (E3).

Révision de théorie/croyances (le voisin le plus proche). `reviseOnBlame` relève de la révision de théorie d’une base de règles *apprise* : EITHER [24] et FORTE [29] révisent des théories Horn sur contre-exemples ; la révision d’un ensemble de croyances est l’AGM [3], et le moniteur d’exécution s’inscrit dans la tradition du raisonnement défaisable / non-monotone [27]. Notre apport est opérationnel : attacher la révision à une bibliothèque de méthodes *typée, composable, canonicalisable* ; nous mesurons cette étape de révision sous dérive récurrente en E8 (§4.9).

Maintien de la vérité. Un JTMS [11, 10].

Exécution durable / IVM. Un cas est un marquage 1-sûr sur un réseau de workflow [32] ; l’exécuteur durable est un artefact connu [4, 31]. DBSP [8]/Materialize maintiennent des *valeurs* ; nous ne revendiquons pas la nouveauté pour *recupérer* sur dérive — la différence est que notre invalidation est *dérivée d’un contrat déclaratif* (re-vérifier la post ; blâmer ; spécialiser). Le jeu chaud et la reconstruction du seul maillon affecté (§4.8) s’apparentent au calcul fonctionnel auto-ajustable [2] et à la reconstruction incrémentale des systèmes de build [22].

6 Menaces à la validité

Simulateur vs réel. Le simulateur retire l’erreur du modèle pour isoler le *mécanisme* ; l’E2 réel confirme l’ordre. Le simulateur ne prédit pas l’exactitude absolue en réel ; exécuter plus de bras en réel est un travail futur. L’exactitude à la dérive est mesurée sur l’oracle déterministe ; les exécutions réelles confirment l’ordre des appels et du temps (le modèle réel est imparfait à petit N , nous ne lisons donc pas la dérive par enregistrement sur lui).

K1-couverture paramétrée. P4 *fixe* la fraction typée ; les affirmations non circulaires sont la *forme* (un gradient), l’universalité de la sûreté, et la non-sûreté de l’amortissement glouton. La fraction absolue d’une charge de production est dépendante du domaine.

Force des références. RAG/CBR/Compétence sont nos implémentations ; le bras Invalidant isole « possède un crochet » de « possède un contrat typé », d’où l’affirmation précise. Une RAG/CBR à invalidation-événementielle ajustée est essentiellement l’Invalidant. Les systèmes *nommés* sont évalués en E6 (§4.7) : des ré-implémentations minimales fidèles de chaque mécanisme (pas d’édition mémoire réellement pilotée par le LLM, ni modèle de récompense appris, ni communautés Leiden), chacune dans sa configuration la plus favorable et *charitable* — des bornes supérieures du comportement à la dérive, non des hommes de paille. De façon symétrique, notre STRUCT n’est pas un stub : une réalisation sur le vrai moteur (rétractation JTMS *ensure-gated* au-dessus du cache de dérivation) reproduit son coin en stub et en réel (9 appels sur le $N = 32$ réel) et la bibliothèque chaude survit à un redémarrage à zéro appel — E6 confronte les ré-implémentations des systèmes nommés au *vrai* moteur, pas stub-contre-stub.

Nouveauté / positionnement. Aucun mécanisme n’est nouveau ; le travail est une *composition*. Nous le positionnons comme une unification opérationnelle, non un nouvel algorithme.

Échelle et étendue. Mécanismes à N modeste sur deux domaines synthétiques ; E5 est un coût de bookkeeping. Le tête-à-tête face aux mémoires d’agents modernes est désormais E6 (§4.7, ré-implémentations minimales fidèles) ; un corpus réel sur un exécuteur durable et une comparaison face aux systèmes *déployés* restent des travaux futurs.

Sûreté éventuelle, non statique. L’applicabilité d’une méthode apprise est indécidable en général (Rice) ; la garantie est une sûreté éventuelle via un moniteur d’exécution porteur au-dessus d’une barrière saine-incomplète.

Moteur unique / auteur unique. Une reproduction indépendante renforcerait les affirmations structurelles.

7 Conclusion

La mémoire par rappel seul ne sait pas désapprendre ; les bibliothèques statiques sont saines mais figées. Une bibliothèque apprise de méthodes typées dotée d’un **contrat d’exécution défaisable** obtient les deux : elle amortit et compose sur la structure typée, et lorsqu’une prémisse dérive elle *rétracte avec blâme et révisé* — récupérant l’exactitude là où la recherche, le CBR et les bibliothèques de compétences servent du périmé. Nos expériences l’isolent : le rappel seul ne récupère pas ; un crochet d’invalidation le peut ; un contrat typé *déclaratif* fournit cette récupération de façon sélective, générale et sûre-en-composition, à contexte borné, sur un moteur réel et confirmé en réel — borné par une fraction canonicalisable elle-même frontière de sûreté. Chaque mécanisme relève de l’état de l’art ; la contribution est leur composition en une représentation typée unique où amortissement, vérification de composition et désapprentissage coïncident. C’est, délibérément, une synthèse d’ingénierie avec une propriété émergente testable — un désapprentissage principal et sélectif — plutôt qu’un nouvel algorithme.

Code & reproductibilité. Le moteur et un artefact d’expérience autonome sont publics sur github.com/9pings/skynet-graph — `artifact/paper-dll/` : mécanismes de base (`workload.js`, `arms.js`, `harness.js`, `e1-transfer.js`, `e3-compose.js`, `p4-coverage.js`, `scale.js`, `measure-e2-live.js`, `F6-transfer.js`), le tête-à-tête E6 (`named-arms.js`, `struct-real.js`, `measure-named-h2h.js`), la suite E7 composition-à-la-dérive (`composed-*.js`, `struct-real-composed.js`, `durable-composed.js`, `chain-depth.js`) et la révision E8 (`revise.js`), avec la suite déterministe `tests/integration/paper-*.test.js` (`npm test`). Les exécutions en réel utilisent un endpoint local compatible OpenAI servant Qwen3.6-27B (Q2_K_XL, MTP). Sous licence AGPL-3.0-or-later.

Références

- [1] A. Aamodt, E. Plaza. Case-Based Reasoning: Foundational Issues, Methodological Variations, and System Approaches. *AI Communications* 7(1):39–59, 1994.
- [2] U. A. Acar, G. E. Blelloch, R. Harper. Adaptive Functional Programming. *POPL* 2002, 247–259.
- [3] C. E. Alchourrón, P. Gärdenfors, D. Makinson. On the Logic of Theory Change. *J. Symbolic Logic* 50(2):510–530, 1985.
- [4] AWS. Step Functions Distributed Map. 2022.
- [5] J. Bader, J. Aldrich, É. Tanter. Gradual Program Verification. *VMCAI* 2018, LNCS 10747:25–46.
- [6] L. Bourtole, V. Chandrasekaran, C. A. Choquette-Choo, H. Jia, A. Travers, B. Zhang, D. Lie, N. Papernot. Machine Unlearning. *IEEE S&P* 2021, 141–159.
- [7] M. Bowers et al. Top-Down Synthesis for Library Learning. *POPL* 2023; PACMPL 7(POPL).
- [8] M. Budiu et al. DBSP: Automatic Incremental View Maintenance for Rich Query Languages. *PVLDB* 16(7):1601–1614, 2023.
- [9] B. Courcelle. The Monadic Second-Order Logic of Graphs I. *Inf. Comput.* 85(1):12–75, 1990.
- [10] J. de Kleer. An Assumption-based TMS. *Artificial Intelligence* 28(2):127–162, 1986.

- [11] J. Doyle. A Truth Maintenance System. *Artificial Intelligence* 12(3):231–272, 1979.
- [12] F. Drewes, H.-J. Kreowski, A. Habel. Hyperedge Replacement Graph Grammars. In *Handbook of Graph Grammars* Vol. 1, World Scientific, 95–162, 1997.
- [13] D. Edge et al. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130, 2024.
- [14] K. Ellis et al. DreamCoder. *PLDI* 2021.
- [15] K. Erol, J. Hendler, D. S. Nau. UMCP: A Sound and Complete Procedure for HTN Planning. *AIPS* 1994.
- [16] K. Erol, J. Hendler, D. S. Nau. Complexity Results for HTN Planning. *Ann. Math. AI* 18:69–93, 1996.
- [17] R. B. Findler, M. Felleisen. Contracts for Higher-Order Functions. *ICFP* 2002, 48–59.
- [18] A. Habel. Hyperedge Replacement: Grammars and Languages. LNCS 643, Springer, 1992.
- [19] P. Lewis et al. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *NeurIPS* 2020.
- [20] J. McCarthy, P. J. Hayes. Some Philosophical Problems from the Standpoint of AI. *Machine Intelligence* 4, 1969.
- [21] T. M. Mitchell, R. M. Keller, S. T. Kedar-Cabelli. Explanation-Based Generalization: A Unifying View. *Machine Learning* 1(1):47–80, 1986.
- [22] A. Mokhov, N. Mitchell, S. Peyton Jones. Build Systems à la Carte. *Proc. ACM Program. Lang.* 2(ICFP), Art. 79, 2018.
- [23] P. W. O’Hearn, J. C. Reynolds, H. Yang. Local Reasoning about Programs that Alter Data Structures. *CSL* 2001, LNCS 2142:1–19.
- [24] D. Ourston, R. J. Mooney. Theory Refinement Combining Analytical and Empirical Methods. *Artificial Intelligence* 66(2):273–309, 1994.
- [25] C. Packer et al. MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560, 2023.
- [26] G. D. Plotkin. A Note on Inductive Generalization. *Machine Intelligence* 5:153–163, 1970.
- [27] R. Reiter. A Logic for Default Reasoning. *Artificial Intelligence* 13(1–2):81–132, 1980.
- [28] J. C. Reynolds. Separation Logic. *LICS* 2002, 55–74.
- [29] B. L. Richards, R. J. Mooney. Automated Refinement of First-Order Horn-Clause Domain Theories. *Machine Learning* 19(2):95–131, 1995.
- [30] N. Shinn et al. Reflexion: Language Agents with Verbal Reinforcement Learning. *NeurIPS* 2023.
- [31] A. Skiadopoulos et al. DBOS: A DBMS-oriented Operating System. *PVLDB* 15(1):21–30, 2021.
- [32] W. M. P. van der Aalst. The Application of Petri Nets to Workflow Management. *J. Circuits, Systems and Computers* 8(1):21–66, 1998.

- [33] G. Wang et al. Voyager: An Open-Ended Embodied Agent with Large Language Models. arXiv:2305.16291, 2023.